



CESI
ÉCOLE D'INGÉNIEURS

Breezy

Réseau social léger & scalable — Clone Twitter/X

Livrable : Rapport de projet

Conception et réalisation d'une application web distribuée

Juin 2026

Auteurs :

Marine MAZOU
Stéphane PLATHEY-BADIN
Nolan JOURDAN

Équipe : Groupe 4 — DAD

Formation : Ingénieur en informatique

École : CESI

Année : 2025 — 2026

TABLE OF CONTENTS

Introduction	2
1 Objectifs et attentes du projet	2
1.1 Contexte et problématique	2
1.2 Objectifs clés	2
1.3 Périmètre fonctionnel attendu	3
1.4 Livrables attendus	3
2 Architecture du système	3
2.1 Vue d'ensemble	3
2.2 Justification des choix d'architecture	4
2.3 Modèle de données	5
3 Hiérarchisation des tâches et fonctionnalités	5
3.1 Matrice des rôles et permissions	5
3.2 Priorisation MoSCoW	6
3.3 Répartition des tâches dans l'équipe	7
4 Étapes et ressources mobilisées	7
4.1 Phases du projet	7
4.2 Ressources mobilisées	8
5 Méthodologie	8
5.1 Démarche itérative	8
5.2 Organisation Git	8
5.3 Intégration et déploiement continu	9
6 Interfaces utilisateur — maquettes	9
7 Fonctionnalités principales et mise en œuvre	10
7.1 Authentification et gestion des rôles	10
7.2 Publication et fil d'actualité	10
7.3 Interactions sociales	10
7.4 Notifications et messagerie	10
7.5 Modération	11
7.6 Couche d'intégration front/back	11
8 Axes d'amélioration et évolutions	11
Conclusion	12
Sitographie	13

INTRODUCTION

Ce rapport présente la conception et la réalisation de **Breezy**, un réseau social léger inspiré de Twitter/X, développé dans le cadre du projet de fin de module en ingénierie informatique au CESI. Le document détaille les objectifs poursuivis, l'architecture technique retenue, la méthodologie de travail mise en œuvre par l'équipe, ainsi que les fonctionnalités livrées et leurs perspectives d'évolution.

Le projet a été mené par une équipe de trois personnes aux rôles complémentaires. En tant qu'**intégrateur Full-Stack, ingénieur DevOps et chef de projet**, ma contribution a porté sur la liaison entre le front-end et le back-end, la conteneurisation et le déploiement de la plateforme, ainsi que sur la coordination et le suivi de l'avancement de l'équipe.

1 OBJECTIFS ET ATTENTES DU PROJET

1.1 CONTEXTE ET PROBLÉMATIQUE

Les réseaux sociaux traditionnels sont souvent lourds et gourmands en ressources, ce qui dégrade l'expérience sur les terminaux modestes ou les connexions limitées. Breezy répond à ce constat en proposant un réseau social **léger, réactif et hautement scalable**, conçu selon une approche *mobile-first*, tout en reposant sur une architecture back-end robuste capable de monter en charge.

1.2 OBJECTIFS CLÉS

- **Performance et réactivité** sur l'ensemble des appareils, y compris les environnements à faibles ressources.
- **Architecture distribuée et scalable**, où chaque domaine fonctionnel est isolé et peut évoluer indépendamment.
- **Expérience utilisateur fluide**, pensée d'abord pour le mobile puis déclinée en interface responsive multi-format.
- **Sécurité renforcée**, fondée sur l'authentification par jeton JWT et une gestion stricte des rôles et permissions.

1.3 PÉRIMÈTRE FONCTIONNEL ATTENDU

Le cahier des charges définit **23 fonctionnalités** (notées Fx1 à Fx23), encadrées par une matrice de permissions associant chaque action à un niveau de rôle. Elles couvrent la gestion de compte, la publication de contenu (texte, images, vidéos), les interactions sociales (likes, commentaires, abonnements), la messagerie privée, les notifications, la modération et le confort d'usage (multi-langues, thème clair/sombre).

1.4 LIVRABLES ATTENDUS

- Une application web fonctionnelle, conteneurisée et déployable en une commande.
- Un code source versionné, organisé en monorepo et documenté.
- Le présent rapport de projet et le support de maintenance.

2 ARCHITECTURE DU SYSTÈME

2.1 VUE D'ENSEMBLE

L'application repose sur une **architecture distribuée en microservices**, orchestrée par Docker. Le client web (Next.js) communique exclusivement à travers une **passerelle API (Nginx)** qui assure le routage, la gestion du CORS, la limitation de débit et la distribution des médias statiques. Chaque microservice est responsable d'un domaine métier et persiste ses données dans la base la plus adaptée à sa nature.

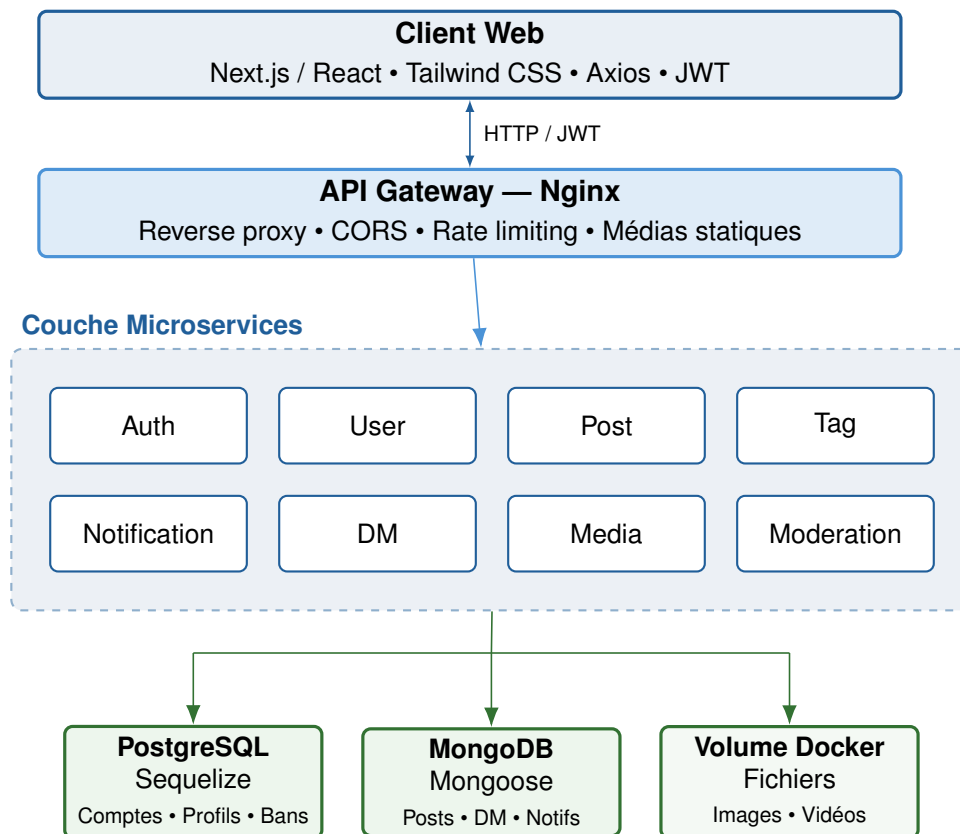


Figure 1: Architecture microservices de Breezy : client, passerelle Nginx, huit services métier et persistance polyglotte (PostgreSQL, MongoDB, volume de médias).

2.2 JUSTIFICATION DES CHOIX D'ARCHITECTURE

Microservices. Le découpage en huit services indépendants (Auth, User, Post, Tag, Notification, DM, Media, Moderation) procure l'isolation des pannes, le déploiement indépendant et une montée en charge ciblée sur les services les plus sollicités.

Passerelle API. Nginx centralise les préoccupations transverses (sécurité, CORS, limitation de débit, journalisation) et masque la topologie interne au client, qui ne connaît qu'un point d'entrée unique.

Persistance polyglotte. Les données relationnelles fortement structurées (comptes, profils, abonnements, bannissements) sont confiées à **PostgreSQL** via Sequelize, tandis que les données documentaires à fort volume et au schéma souple (posts, commentaires imbriqués, tags, messages, notifications, signalements) sont stockées dans **MongoDB** via Mongoose.

2.3 MODÈLE DE DONNÉES

Moteur	Entité	Champs principaux
PostgreSQL	Users	id, username, email, password_hash, role, is_validated
PostgreSQL	Profiles	user_id, display_name, bio, avatar_url, language, theme
PostgreSQL	Followers	follower_id, following_id, created_at
PostgreSQL	Bans	user_id, reason, banned_by, expires_at
MongoDB	Posts	user_id, content, likes[], comments[], tags[], media
MongoDB	Notifications	recipient_id, sender_id, type, post_id, is_read
MongoDB	DirectMessages	conversation_id, sender_id, recipient_id, message_text
MongoDB	Reports	reported_by, target_type, target_id, reason, status

Table 1: Répartition des données entre PostgreSQL et MongoDB.

3 HIÉRARCHISATION DES TÂCHES ET FONCTIONNALITÉS

3.1 MATRICE DES RÔLES ET PERMISSIONS

L'accès aux fonctionnalités est gouverné par une hiérarchie de quatre rôles : **Visiteur** < **Utilisateur** < **Modérateur** < **Administrateur**. Cette matrice constitue la colonne vertébrale fonctionnelle du projet et a guidé la priorisation du développement.

#	Fonctionnalité	Vis.	Util.	Mod.	Admin
Fx1	Création de compte avec validation	✓	—	—	✓
Fx2	Authentification sécurisée (JWT)	—	✓	✓	✓
Fx3	Publication de messages (280 car.)	—	✓	✓	✓
Fx4–5	Profil & flux chronologique	—	✓	✓	✓
Fx6–8	Liker, commenter, répondre	—	✓	✓	✓
Fx9–11	Suivre, profils, posts d'un user	—	✓	✓	✓
Fx12–13	Tags et recherche par tag	—	✓	✓	✓
Fx14–16	Notifications (mention/like/follow)	—	✓	✓	✓
Fx17	Messages privés	—	✓	✓	✓
Fx18–19	Images et vidéos dans les messages	—	✓	✓	✓
Fx20	Signalement de contenu	—	✓	✓	✓
Fx21	Suspension / bannissement	—	—	✓	✓
Fx22	Interface multi-langues	—	✓	✓	✓
Fx23	Thème sombre / clair	✓	✓	✓	✓

Table 2: Matrice des permissions par rôle (23 fonctionnalités).

3.2 PRIORISATION MOSCOW

Les fonctionnalités ont été hiérarchisées selon la méthode MoSCoW afin de sécuriser un socle livrable avant d'enrichir le périmètre :

- **Must have** — authentification JWT, publication, feed, likes, commentaires, abonnements, profils (cœur du réseau social).
- **Should have** — tags et recherche, notifications, messages privés, médias images.
- **Could have** — vidéos, modération avancée, multi-langues, thème clair/sombre.
- **Won't have (pour cette itération)** — appels audio/vidéo, recommandation algorithmique du fil.

3.3 RÉPARTITION DES TÂCHES DANS L'ÉQUIPE

Rôle	Responsabilités principales
Membre 1 — Back-end	Microservices, modèles de données, API REST, tests unitaires et d'intégration, sécurité applicative.
Membre 2 — Front-end	Interfaces Next.js, composants, design system Tailwind, maquettes et ergonomie mobile-first.
Membre 3 — Full-Stack / DevOps / Chef de projet	Liaison front/back, conteneurisation Docker, passerelle Nginx, déploiement, coordination et suivi de l'avancement.

Table 3: Répartition des responsabilités au sein de l'équipe.

4 ÉTAPES ET RESSOURCES MOBILISÉES

4.1 PHASES DU PROJET

1. **Cadrage** — analyse du besoin, définition des 23 fonctionnalités et de la matrice de permissions, choix de la stack.
2. **Conception** — modélisation des données, découpage en microservices, définition des contrats d'API.
3. **Développement back-end** — implémentation des services, des modèles Sequelize/Mongoose et des routes REST.
4. **Développement front-end** — construction des écrans, du design system et du rendu responsive.
5. **Intégration** — liaison front/back via la couche de services Axios, gestion du JWT, harmonisation des contrats.
6. **Déploiement & recette** — conteneurisation, orchestration Docker Compose, jeu de données de démonstration, tests de bout en bout.

4.2 RESSOURCES MOBILISÉES

Catégorie	Détail
Humaines	Équipe de 3 développeurs aux rôles complémentaires.
Langages	TypeScript (back), JavaScript / JSX (front).
Frameworks	Express.js, Next.js / React, Tailwind CSS.
Données	PostgreSQL 15 + Sequelize, MongoDB 6 + Mongoose.
Infrastructure	Docker, Docker Compose, Nginx, npm Workspaces.
Sécurité	JWT (HS256), bcrypt, RBAC, en-têtes de sécurité Nginx.
Outillage	Git / GitHub, GitHub Actions (CI), Adminer, mongo-express.

Table 4: Ressources techniques et humaines mobilisées.

5 MÉTHODOLOGIE

5.1 DÉMARCHE ITÉRATIVE

Le projet a suivi une démarche **agile et itérative** : le périmètre a été livré par incréments fonctionnels, chaque itération ajoutant un ensemble cohérent de fonctionnalités testables. Cette approche a permis de disposer rapidement d'un socle démontrable et d'absorber les ajustements (vidéos, gestion fine des rôles) sans remettre en cause l'architecture.

5.2 ORGANISATION GIT

Le code est versionné dans un **monorepo** géré avec npm Workspaces. Le travail s'appuie sur un modèle de branches par fonctionnalité :

- `main` — version stable et déployable.
- `dev` — branche d'intégration consolidant le back-end et le front-end.
- `feat/*` — une branche par fonctionnalité ou chantier d'intégration.

Les messages de commit respectent la convention **Conventional Commits** (`feat`, `fix`, `chore`, `docs`...), ce qui rend l'historique lisible et facilite la relecture entre pairs via les *pull requests*.

5.3 INTÉGRATION ET DÉPLOIEMENT CONTINUS

Une chaîne **CI** (GitHub Actions) exécute la compilation du package partagé puis les tests unitaires et d'intégration de chaque service. Le déploiement local est entièrement automatisé : une seule commande `docker compose up` démarre les bases de données, les huit services, la passerelle Nginx et le front-end.

6 INTERFACES UTILISATEUR — MAQUETTES

L'interface a été conçue *mobile-first* puis déclinée en version responsive « trois colonnes » sur grand écran, à la manière de X. Les wireframes ci-dessous synthétisent les deux dispositions principales.

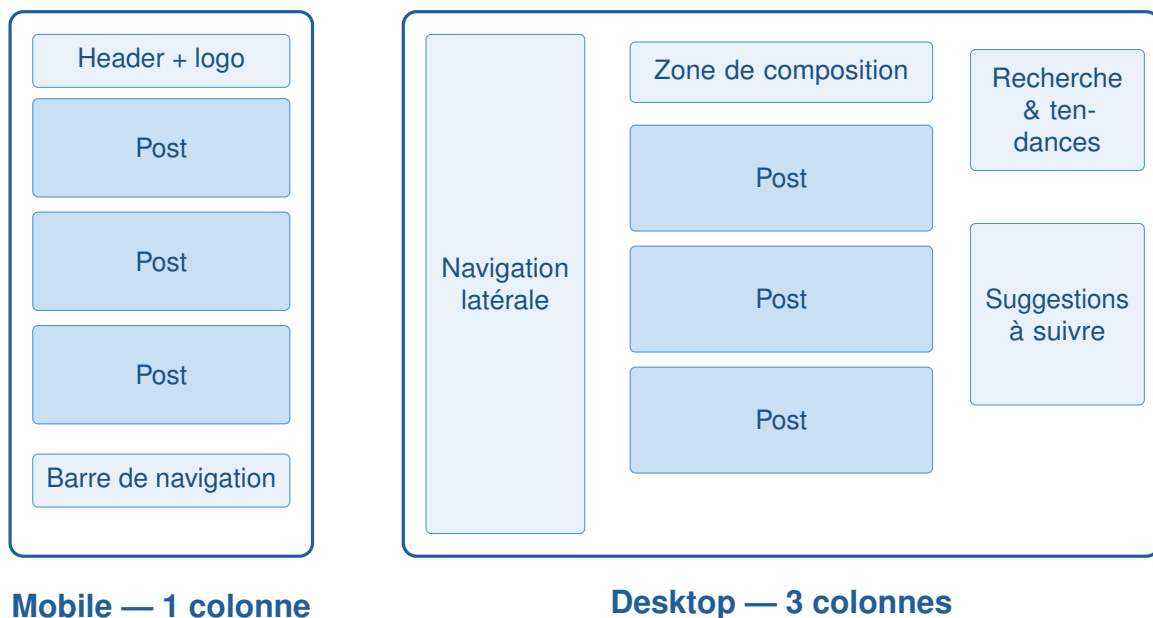


Figure 2: Wireframes des dispositions mobile (une colonne, navigation basse) et bureau (trois colonnes : navigation, fil central, tendances).

Principes d'ergonomie. La navigation principale passe d'une barre inférieure sur mobile à une barre latérale sur bureau ; le fil central reste l'élément focal ; la colonne de droite (recherche, tendances, suggestions) n'apparaît qu'à partir des grands écrans. Un thème clair/sombre et un sélecteur de langue (FR/EN/ES) sont accessibles depuis l'interface.

7 FONCTIONNALITÉS PRINCIPALES ET MISE EN ŒUVRE

7.1 AUTHENTIFICATION ET GESTION DES RÔLES

L'inscription crée un compte en attente de validation ; la connexion renvoie un **jeton JWT** (signé HS256, valable une heure) transmis ensuite dans l'en-tête `Authorization: Bearer`. Côté front, le jeton est conservé et injecté automatiquement dans chaque requête Axios. Le contrôle d'accès (RBAC) est appliqué côté serveur par un middleware d'authentification, et reflété côté client en masquant les actions non autorisées (par exemple, le bouton de modération n'est visible que pour les modérateurs et administrateurs).

7.2 PUBLICATION ET FIL D'ACTUALITÉ

La création de post accepte un texte (280 caractères), des tags, et un média optionnel **image ou vidéo** : le fichier est d'abord envoyé au service Media, puis l'URL retournée est attachée au post. Le fil chronologique agrège les posts des comptes suivis ; un *mapper* dédié transforme le document MongoDB en modèle d'affichage et résout l'auteur (identifiant → nom).

7.3 INTERACTIONS SOCIALES

Les likes utilisent une **mise à jour optimiste** : l'interface bascule immédiatement puis se recalcule sur la réponse du serveur (ou annule en cas d'erreur). Commentaires et réponses imbriquées sont stockés directement dans le document du post. Le suivi d'un utilisateur met à jour la table relationnelle des abonnements et déclenche une notification.

7.4 NOTIFICATIONS ET MESSAGERIE

Les notifications (mentions, likes, nouveaux abonnés) sont remontées par un hook partagé entre l'en-tête et la barre latérale, qui résout les expéditeurs et formate les libellés. La messagerie privée regroupe les échanges par conversation et expose le nombre de messages non lus.

7.5 MODÉRATION

Tout utilisateur peut **signaler** un contenu ; les signalements alimentent une file consultable par les modérateurs, qui peuvent les résoudre et, le cas échéant, **suspendre ou bannir** un compte. L'accès à l'espace de modération est protégé côté serveur et masqué côté client aux rôles non habilités.

7.6 COUCHE D'INTÉGRATION FRONT/BACK

La liaison entre les deux mondes constitue le cœur de ma contribution. Elle repose sur :

- une **couche de services Axios** centralisant les appels, la base URL de la passerelle et l'injection du jeton ;
- une **enveloppe de réponse standardisée** et des *mappers* purs convertissant les modèles back-end en modèles d'affichage ;
- la **passerelle Nginx** réglée pour le CORS, la limitation de débit adaptée à une SPA et la diffusion publique des médias ;
- un **déploiement Docker Compose** complet accompagné d'un jeu de données de démonstration (utilisateurs liés, posts avec tags, médias et notifications) et d'outils web d'inspection des bases (Adminer, mongo-express).

8 AXES D'AMÉLIORATION ET ÉVOLUTIONS

- **Temps réel** — diffusion des notifications et des messages via WebSocket plutôt que par interrogation périodique.
- **Recherche avancée** — moteur full-text (posts, utilisateurs, tags) et fil de tendances calculé dynamiquement.
- **Observabilité** — journalisation centralisée, métriques et traces distribuées sur l'ensemble des microservices.
- **Mise à l'échelle** — orchestration Kubernetes, réplication des bases et mise en cache (Redis) des flux les plus consultés.
- **Sécurité** — rafraîchissement des jetons (refresh tokens), HTTPS de bout en bout et audit des dépendances automatisé.
- **Qualité** — couverture de tests end-to-end et tests de charge pour valider la promesse de scalabilité.

CONCLUSION

Breezy démontre qu'un réseau social complet peut être bâti sur une architecture microservices conteneurisée, sécurisée et déployable en une commande, tout en respectant une exigence de légèreté et de réactivité. Les 23 fonctionnalités prévues ont été couvertes, de l'authentification à la modération, en passant par la publication de médias et les interactions sociales.

Au-delà du résultat technique, ce projet a renforcé des compétences clés d'intégration Full-Stack, de pratiques DevOps (conteneurisation, passerelle API, déploiement automatisé) et de conduite de projet (organisation Git, priorisation, coordination d'équipe). Les axes d'évolution identifiés — temps réel, observabilité et mise à l'échelle — ouvrent des perspectives concrètes pour transformer ce prototype abouti en service de production.

SITOGRAFIE

- Documentation Next.js — <https://nextjs.org/docs>
- Documentation Express.js — <https://expressjs.com>
- Documentation Docker — <https://docs.docker.com>
- Documentation PostgreSQL / Sequelize — <https://sequelize.org>
- Documentation MongoDB / Mongoose — <https://mongoosejs.com>
- Spécification Conventional Commits — <https://www.conventionalcommits.org>